

Python Algorithms: Pocket Reference

Searching & Sorting in Object-Oriented, Security-Scripting Style

Scope: Sections 7–11 from `nesc.txt`

Goal: Learn the *idea*, the *workflow*, and a *minimal working example* for each algorithm.

Style: Object-oriented Python, with security-scripting flavoured examples (log lines, ports, IPs, timestamps).

1. The Big Picture

Task	What it does	Real-life security example
Searching	Find something in a collection	Is port 22 open in a scan list? Find a malicious IP in logs.
Sorting	Put a collection in order	Sort firewall rules by port, sort alerts by severity/risk score.

Mental model

Input (list) → Algorithm (steps) (logic/workflow) → Output (found index / sorted list)

2. Searching Algorithms

2.1 Linear Search — “Check every item, one by one”

When to use: Small or unsorted lists.

Workflow:

```
for each item in list:
    if item == target:
        return position
return not found

class LinearSearcher:
    def __init__(self, items):
        self.items = items

    def find(self, target):
        for index, item in enumerate(self.items):
            if item == target:
                return index
        return -1 # not found

# Example: find a suspicious port
ports = [80, 443, 22, 8080, 3389]
```

```
search = LinearSearcher(ports)
print(search.find(22))    # 2
print(search.find(53))    # -1
```

Complexity: $O(n)$ — worst case, looks at every element.

2.2 Binary Search — “Halve the sorted list repeatedly”

When to use: The list must already be sorted.

Workflow:

```
while low <= high:
    mid = (low + high) // 2
    if target == mid_value: return mid
    if target < mid_value: high = mid - 1
    else: low = mid + 1
return not found

class BinarySearcher:
    def __init__(self, items):
        # Binary search REQUIRES sorted data
        self.items = sorted(items)

    def find(self, target):
        low, high = 0, len(self.items) - 1
        while low <= high:
            mid = (low + high) // 2
            if self.items[mid] == target:
                return mid
            elif target < self.items[mid]:
                high = mid - 1
            else:
                low = mid + 1
        return -1

# Example: find a banned IP in a sorted blacklist
ips = ["10.0.0.5", "10.0.0.12", "10.0.0.20", "10.0.0.99"]
search = BinarySearcher(ips)
print(search.find("10.0.0.20")) # 2
print(search.find("10.0.0.50")) # -1
```

Complexity: $O(\log n)$ — very fast for big sorted lists.

3. Sorting Algorithms

One sentence each

Algorithm	Core idea
Bubble	Repeatedly swap neighbours if they are out of order.
Selection	Pick the smallest remaining item and place it next.

Algorithm	Core idea
Insertion	Take one card at a time and insert it into the right spot.
Merge	Split in half, sort each half, merge them back together.
Quick	Pick a pivot; smaller items go left, larger go right; repeat.
Heap	Use a binary heap structure to pull the largest item repeatedly.
Bucket	Scatter items into buckets, sort each bucket, gather them.
Radix	Sort digit by digit (or byte by byte).

3.1 Bubble Sort — “Big values bubble to the end”

```
class BubbleSorter:
    def sort(self, items):
        arr = items.copy()
        n = len(arr)
        for i in range(n):
            for j in range(0, n - i - 1):
                if arr[j] > arr[j + 1]:
                    arr[j], arr[j + 1] = arr[j + 1], arr[j]
        return arr

risk_scores = [3, 1, 4, 1, 5, 9, 2]
print(BubbleSorter().sort(risk_scores))
# [1, 1, 2, 3, 4, 5, 9]
```

Complexity: $O(n^2)$ — simple but slow for large data.

3.2 Selection Sort — “Find the minimum, place it, repeat”

```
class SelectionSorter:
    def sort(self, items):
        arr = items.copy()
        n = len(arr)
        for i in range(n):
            min_idx = i
            for j in range(i + 1, n):
                if arr[j] < arr[min_idx]:
                    min_idx = j
            arr[i], arr[min_idx] = arr[min_idx], arr[i]
        return arr

ports = [8080, 22, 443, 80, 3306]
print(SelectionSorter().sort(ports))
# [22, 80, 443, 3306, 8080]
```

Complexity: $O(n^2)$ — fewer swaps than bubble, but still slow.

3.3 Insertion Sort — “Insert each new item in its correct place”

```
class InsertionSorter:
    def sort(self, items):
        arr = items.copy()
        for i in range(1, len(arr)):
            key = arr[i]
            j = i - 1
            while j >= 0 and arr[j] > key:
                arr[j + 1] = arr[j]
                j -= 1
            arr[j + 1] = key
        return arr

severities = ["low", "critical", "medium", "low", "high"]
# Map to numbers for sorting, then map back
rank = {"low": 1, "medium": 2, "high": 3, "critical": 4}
sorted_nums = InsertionSorter().sort([rank[s] for s in severities])
back = {v: k for k, v in rank.items()}
print([back[n] for n in sorted_nums])
# ['low', 'low', 'medium', 'high', 'critical']
```

Complexity: $O(n^2)$ worst, but $O(n)$ best for nearly-sorted data.

3.4 Merge Sort — “Divide, conquer, merge”

```
class MergeSorter:
    def sort(self, items):
        arr = items.copy()
        self._merge_sort(arr, 0, len(arr) - 1)
        return arr

    def _merge_sort(self, arr, left, right):
        if left < right:
            mid = (left + right) // 2
            self._merge_sort(arr, left, mid)
            self._merge_sort(arr, mid + 1, right)
            self._merge(arr, left, mid, right)

    def _merge(self, arr, left, mid, right):
        left_part = arr[left:mid + 1]
        right_part = arr[mid + 1:right + 1]
        i = j = 0
        k = left
        while i < len(left_part) and j < len(right_part):
            if left_part[i] <= right_part[j]:
                arr[k] = left_part[i]
                i += 1
            else:
                arr[k] = right_part[j]
                j += 1
            k += 1
        while i < len(left_part):
            arr[k] = left_part[i]
            i += 1
            k += 1
        while j < len(right_part):
            arr[k] = right_part[j]
            j += 1
            k += 1
```

```

        j += 1
        k += 1

events = [1620, 1500, 1800, 1430, 1700] # timestamps
print(MergeSorter().sort(events))
# [1430, 1500, 1620, 1700, 1800]

```

Complexity: $O(n \log n)$ — consistently fast; uses extra memory.

3.5 Quick Sort — “Pick a pivot, partition, recurse”

```

class QuickSorter:
    def sort(self, items):
        arr = items.copy()
        self._quick_sort(arr, 0, len(arr) - 1)
        return arr

    def _quick_sort(self, arr, low, high):
        if low < high:
            pivot_index = self._partition(arr, low, high)
            self._quick_sort(arr, low, pivot_index - 1)
            self._quick_sort(arr, pivot_index + 1, high)

    def _partition(self, arr, low, high):
        pivot = arr[high]
        i = low - 1
        for j in range(low, high):
            if arr[j] <= pivot:
                i += 1
                arr[i], arr[j] = arr[j], arr[i]
        arr[i + 1], arr[high] = arr[high], arr[i + 1]
        return i + 1

packet_sizes = [512, 64, 1500, 128, 1024]
print(QuickSorter().sort(packet_sizes))
# [64, 128, 512, 1024, 1500]

```

Complexity: $O(n \log n)$ average, $O(n^2)$ worst (rare with good pivots).

3.6 Heap Sort — “Turn the list into a heap, pop the max repeatedly”

A **heap** is a binary tree where the parent is always larger (max-heap) than its children.

```

class HeapSorter:
    def sort(self, items):
        arr = items.copy()
        n = len(arr)

        # Build max heap
        for i in range(n // 2 - 1, -1, -1):
            self._heapify(arr, n, i)

        # Extract elements one by one
        for i in range(n - 1, 0, -1):
            arr[0], arr[i] = arr[i], arr[0]
            self._heapify(arr, i, 0)
        return arr

```

```

def _heapify(self, arr, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2
    if left < n and arr[left] > arr[largest]:
        largest = left
    if right < n and arr[right] > arr[largest]:
        largest = right
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        self._heapify(arr, n, largest)

priorities = [2, 5, 1, 8, 3]
print(HeapSorter().sort(priorities))
# [1, 2, 3, 5, 8]

```

Complexity: $O(n \log n)$ — in-place, no recursion stack explosion.

3.7 Bucket Sort — “Put items in buckets, sort buckets, combine”

```

class BucketSorter:
    def sort(self, items, bucket_count=5):
        if not items:
            return []
        arr = items.copy()
        min_val, max_val = min(arr), max(arr)
        buckets = [[] for _ in range(bucket_count)]

        for num in arr:
            idx = (num - min_val) * (bucket_count - 1) // (max_val - min_val)
            buckets[idx].append(num)

        result = []
        for bucket in buckets:
            result.extend(sorted(bucket)) # could use insertion sort here
        return result

latency_ms = [120, 45, 300, 90, 30, 250]
print(BucketSorter().sort(latency_ms))
# [30, 45, 90, 120, 250, 300]

```

Complexity: $O(n)$ average when data is evenly spread; $O(n^2)$ worst.

3.8 Radix Sort — “Sort by each digit, least significant first”

```

class RadixSorter:
    def sort(self, items):
        arr = items.copy()
        max_num = max(arr) if arr else 0
        exp = 1
        while max_num // exp > 0:
            self._counting_sort_by_digit(arr, exp)
            exp *= 10
        return arr

    def _counting_sort_by_digit(self, arr, exp):
        output = [0] * len(arr)
        count = [0] * 10

```

```

for num in arr:
    digit = (num // exp) % 10
    count[digit] += 1

for i in range(1, 10):
    count[i] += count[i - 1]

for i in range(len(arr) - 1, -1, -1):
    digit = (arr[i] // exp) % 10
    output[count[digit] - 1] = arr[i]
    count[digit] -= 1

for i in range(len(arr)):
    arr[i] = output[i]

ids = [170, 45, 75, 90, 802, 24, 2, 66]
print(RadixSorter().sort(ids))
# [2, 24, 45, 66, 75, 90, 170, 802]

```

Complexity: $O(nk)$ where k = number of digits — great for integers.

4. Algorithm Comparison Table

Algorithm	Best	Average	Worst	Space	Stable	Notes
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Only for teaching.
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Fewer swaps than bubble.
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Great for small/nearly sorted data.
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	Reliable, predictable.
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No*	Fast in practice.
Heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No	Good when memory matters.
Bucket	$O(n+k)$	$O(n+k)$	$O(n^2)$	$O(n+k)$	Yes	Needs uniformly distributed data.
Radix	$O(nk)$	$O(nk)$	$O(nk)$	$O(n+k)$	Yes	Excellent for fixed-width integers.

Stable: Equal items keep their original order.

5. Python Built-in Sorting (Use These in Real Scripts)

`sorted()` — returns a new sorted list

```
ports = [8080, 22, 443, 80]
sorted_ports = sorted(ports)
print(sorted_ports) # [22, 80, 443, 8080]
```

`.sort()` — sorts the list in place

```
alerts = ["medium", "low", "critical", "high"]
rank = {"low": 1, "medium": 2, "high": 3, "critical": 4}

alerts.sort(key=lambda s: rank[s])
print(alerts) # ['low', 'medium', 'high', 'critical']
```

Sort by a custom attribute (OOP example)

```
class Alert:
    def __init__(self, name, severity, timestamp):
        self.name = name
        self.severity = severity
        self.timestamp = timestamp

    def __repr__(self):
        return f"{self.name}({self.severity}, {self.timestamp})"

alerts = [
    Alert("brute-force", "high", 1003),
    Alert("port-scan", "low", 1001),
    Alert("malware", "critical", 1002),
]

# Sort by severity rank
rank = {"low": 1, "medium": 2, "high": 3, "critical": 4}
by_severity = sorted(alerts, key=lambda a: rank[a.severity])
print(by_severity)
# [port-scan(low, 1001), malware(critical, 1002), brute-force(high, 1003)]

# Sort by timestamp
by_time = sorted(alerts, key=lambda a: a.timestamp)
print(by_time)
```

Reverse sort

```
sorted(ports, reverse=True) # [8080, 443, 80, 22]
```

Under the hood

- Python's `list.sort()` and `sorted()` use **Timsort**, a hybrid of merge sort + insertion sort.
- Time: $O(n \log n)$ average/worst.
- Space: $O(n)$.
- **Stable**.

6. Searching with Built-ins

```
ports = [22, 80, 443, 8080]

# Linear-ish index lookup
print(ports.index(443))      # 2
# print(ports.index(53))     # ValueError

# Membership test (fast for sets)
print(443 in ports)         # True
print(53 in ports)         # False

# For big data: use a set for O(1) lookups
banned_ips = {"192.168.1.5", "10.0.0.99", "172.16.0.1"}
print("10.0.0.99" in banned_ips) # True

# Binary search from the standard library
import bisect
sorted_ports = [22, 80, 443, 8080, 8443]
print(bisect.bisect_left(sorted_ports, 443)) # 2
print(bisect.bisect_left(sorted_ports, 53))  # 1 (where to insert)
```

7. Quick Workflow Cheatsheet

When you need to search:

1. Is the data sorted?
 - **Yes** → binary search (`bisect`).
 - **No and small** → linear search / `in` / `.index()`.
 - **No and huge, repeated lookups** → sort first, then binary search, or use a `set/dict`.

When you need to sort:

1. In real code: `sorted()` or `.sort(key=...)`.
2. For learning/exams: know bubble, selection, insertion, merge, quick, heap, bucket, radix.
3. Choose by constraints:
 - Small / nearly sorted → insertion.
 - Guaranteed $O(n \log n)$ → merge or heap.
 - Fast average practice → quick.
 - Integer ranges → radix / bucket.

8. One-File Practice Script

Save as `algo_practice.py` and run it:

```
from python_algorithms_pocket_reference import (
```

```

    LinearSearcher, BinarySearcher,
    BubbleSorter, SelectionSorter, InsertionSorter,
    MergeSorter, QuickSorter, HeapSorter,
    BucketSorter, RadixSorter,
)

if __name__ == "__main__":
    data = [64, 34, 25, 12, 22, 11, 90]
    target = 22

    print("Linear:", LinearSearcher(data).find(target))
    print("Binary:", BinarySearcher(data).find(target))

    print("Bubble:", BubbleSorter().sort(data))
    print("Select:", SelectionSorter().sort(data))
    print("Insert:", InsertionSorter().sort(data))
    print("Merge: ", MergeSorter().sort(data))
    print("Quick: ", QuickSorter().sort(data))
    print("Heap: ", HeapSorter().sort(data))
    print("Bucket:", BucketSorter().sort(data))
    print("Radix: ", RadixSorter().sort(data))

```

Note: importing from a markdown file won't work. Copy the classes into a .py file first.

9. Key Takeaways

- **Searching:** linear = simple; binary = fast but requires sorted data.
- **Sorting:** bubble/selection/insertion are $O(n^2)$ and are mainly for learning.
- **Merge/quick/heap** are $O(n \log n)$ and are the practical heavy lifters.
- **Bucket/radix** are special-purpose and can beat $O(n \log n)$ for certain data.
- **In Python scripts:** use `sorted()`, `.sort()`, `in`, `set`, and `bisect`.

Reference built for the *nesc.txt* sections 7–11.